UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS3014 - Estructuras de Datos Avanzadas

Enteros III
Semana 12

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

27 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Entender por qué un B-tree de ramificación $w^{1/5}$ lleva al tiempo $O(\log_w n)$ si se puede buscar en un nodo en $O(1)$
- Comprimir las llaves de un nodo con *sketches* y corregir la búsqueda cuando q no es una llave
- Calcular el sketch con una multiplicación y comparar con todas las llaves de un nodo con una resta

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Entender por qué un B-tree de ramificación $w^{1/5}$ lleva al tiempo $O(\log_w n)$ si se puede buscar en un nodo en $O(1)$
- Comprimir las llaves de un nodo con *sketches* y corregir la búsqueda cuando q no es una llave
- Calcular el sketch con una multiplicación y comparar con todas las llaves de un nodo con una resta
- Combinar vEB y fusion trees en $O(\sqrt{\lg n})$ y ubicar todo en el panorama de cotas y de ordenamiento de enteros

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dos estructuras para el predecesor

vEB y Y-fast tree cuestan $O(\lg w)$; un árbol de búsqueda, $O(\lg n)$. Para $w = 64$ y $n = 10^9$: 6 contra 30.

¿Y si w es enorme, por ejemplo $w = n^{0,5}$? ¿$\lg w$ sigue siendo bueno?

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Dos estructuras para el predecesor

vEB y Y-fast tree cuestan $O(\lg w)$; un árbol de búsqueda, $O(\lg n)$. Para $w = 64$ y $n = 10^9$: 6 contra 30.

¿Y si w es enorme, por ejemplo $w = n^{0,5}$? ¿$\lg w$ sigue siendo bueno?

Hay un tiempo que *decrece* con w: $\log_w n = \lg n / \lg w$

Con $w = n^{0,5}$, $\log_w n = 2$. La idea es usar los w bits de una palabra para tomar decisiones sobre **muchas** llaves a la vez.

La idea general

Un B-tree con ramificación $w^{1/5}$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Cada nodo tiene hasta $k = w^{1/5}$ llaves

La altura es $\log_{w^{1/5}} n = 5 \log_w n = \Theta(\log_w n)$. Cualquier potencia pequeña de w sirve, y la elegimos para que las cuentas siguientes quepan en una palabra.

¿Cuánto cuesta buscar a q entre las k llaves de un nodo?

Un B-tree con ramificación $w^{1/5}$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Cada nodo tiene hasta $k = w^{1/5}$ llaves

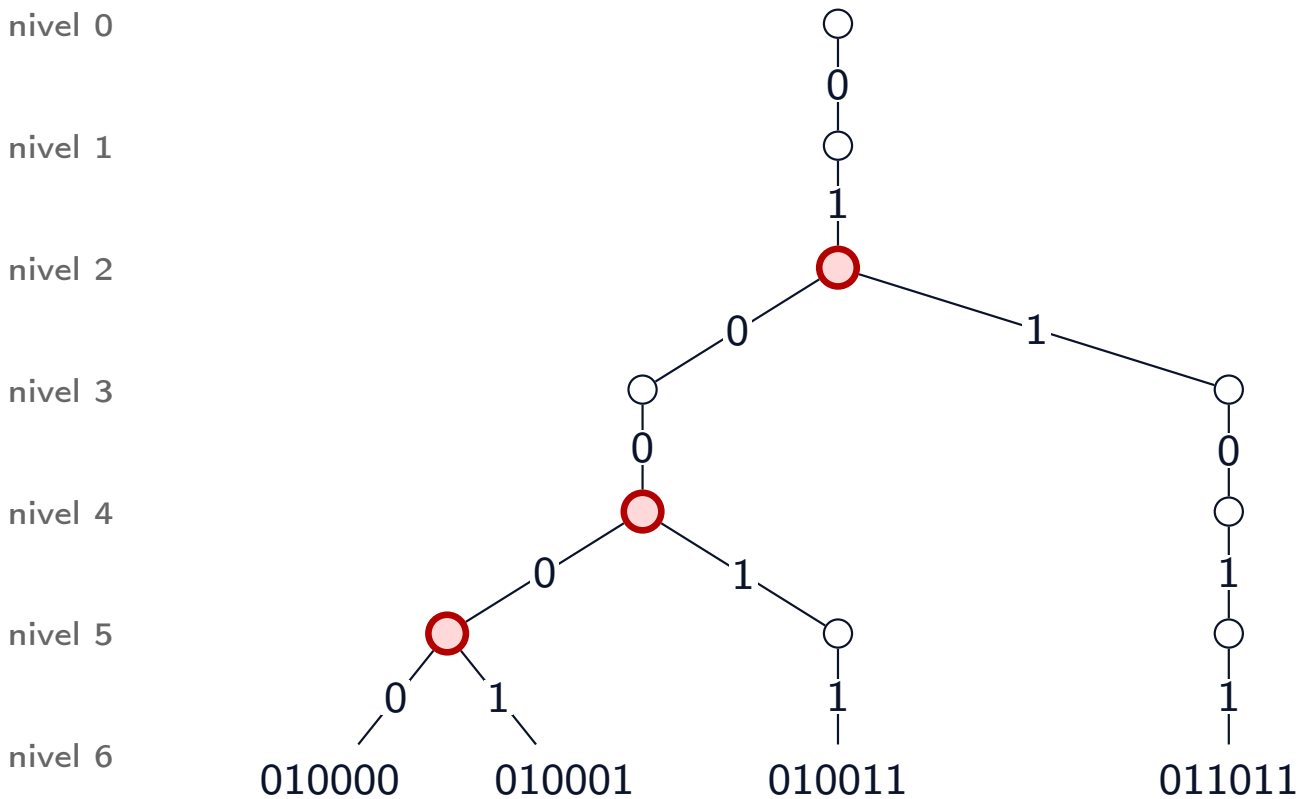
La altura es $\log_{w^{1/5}} n = 5 \log_w n = \Theta(\log_w n)$. Cualquier potencia pequeña de w sirve, y la elegimos para que las cuentas siguientes quepan en una palabra.

¿Cuánto cuesta buscar a q entre las k llaves de un nodo?

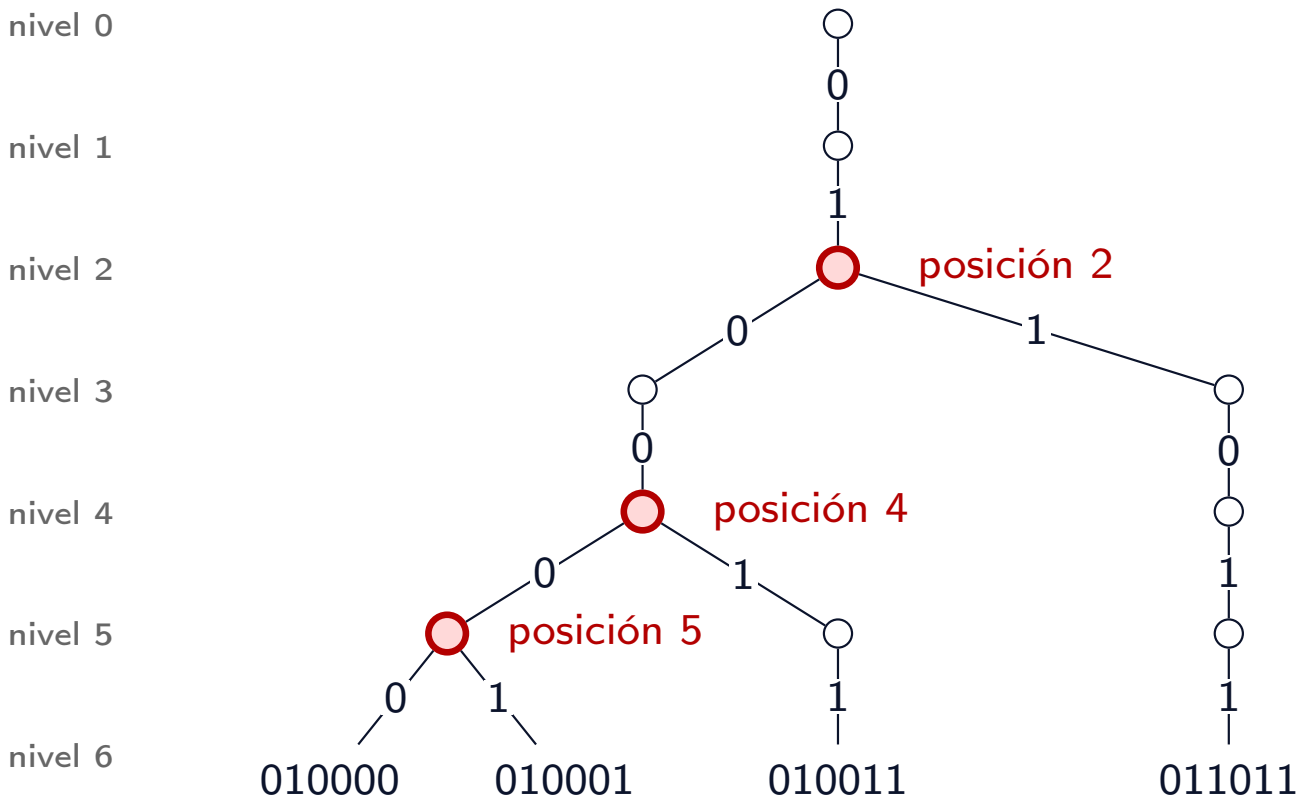
Leerlas ya cuesta más que una palabra

k llaves de w bits son $w^{6/5} > w$ bits. Si cada nodo se resuelve en $O(1)$, el predecesor completo cuesta $O(\log_w n)$: todo el trabajo está en el nodo.

¿Cuántos bits de una llave importan?



¿Cuántos bits de una llave importan?



En rojo, los nodos que ramifican. Con k llaves hay $k-1$: a lo más $k-1$ posiciones de bit (desde la izquierda) distinguen a las llaves.

Sketch perfecto

Definición

Sean $c_0 < c_1 < \cdots < c_{r-1}$ los **exponentes** de los bits donde ramifica el trie ($r \leq k-1$). El sketch de x es el número de r bits

$$\mathrm{sk}(x) = \sum_{j=0}^{r-1} x_{c_j}\, 2^j,$$

con x_c el bit de peso 2^c. Las posiciones $2, 4, 5$ del dibujo (desde la izquierda) son los exponentes $3, 1, 0$.

Sketch perfecto

Definición

Sean $c_0 < c_1 < \cdots < c_{r-1}$ los **exponentes** de los bits donde ramifica el trie ($r \leq k-1$). El sketch de x es el número de r bits

$$\mathrm{sk}(x) = \sum_{j=0}^{r-1} x_{c_j}\, 2^j,$$

con x_c el bit de peso 2^c. Las posiciones $2, 4, 5$ del dibujo (desde la izquierda) son los exponentes $3, 1, 0$.

Llave	Bits $c = \{0, 1, 3\}$	sk
010000	0, 0, 0	000
010001	1, 0, 0	001
010011	1, 1, 0	011
011011	1, 1, 1	111

El sketch conserva el orden entre las llaves

Dos llaves $x < x'$ difieren por primera vez en el bit c. ¿Es c uno de los c_j?

El sketch conserva el orden entre las llaves

Dos llaves $x < x'$ difieren por primera vez en el bit c. ¿Es c uno de los c_j?

Sí: allí ramifica el trie, y por eso $\mathrm{sk}(x) < \mathrm{sk}(x')$

El primer bit donde difieren es el nodo del trie donde se separan: un nodo de ramificación. Más arriba coinciden y, entre los bits que importan, el primero que difiere es justo ese. Entonces $\mathrm{sk}(x_0) < \mathrm{sk}(x_1) < \cdots < \mathrm{sk}(x_{k-1})$.

Todos los sketches caben en una palabra

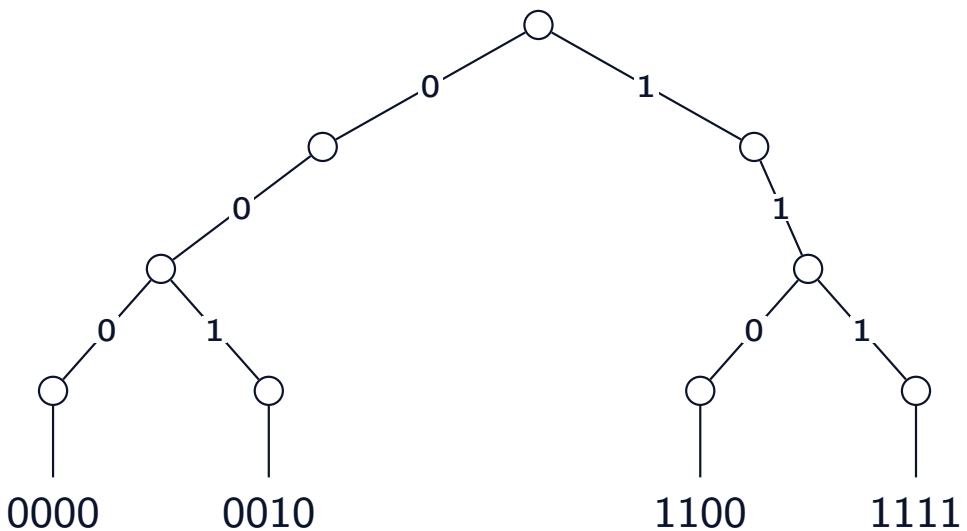
$k \cdot r \leq k^2 = w^{2/5} \ll w$ bits: el nodo guarda un solo número con los sketches de todas sus llaves.

Buscar cuando q no es una llave

El problema

q puede no ser una llave

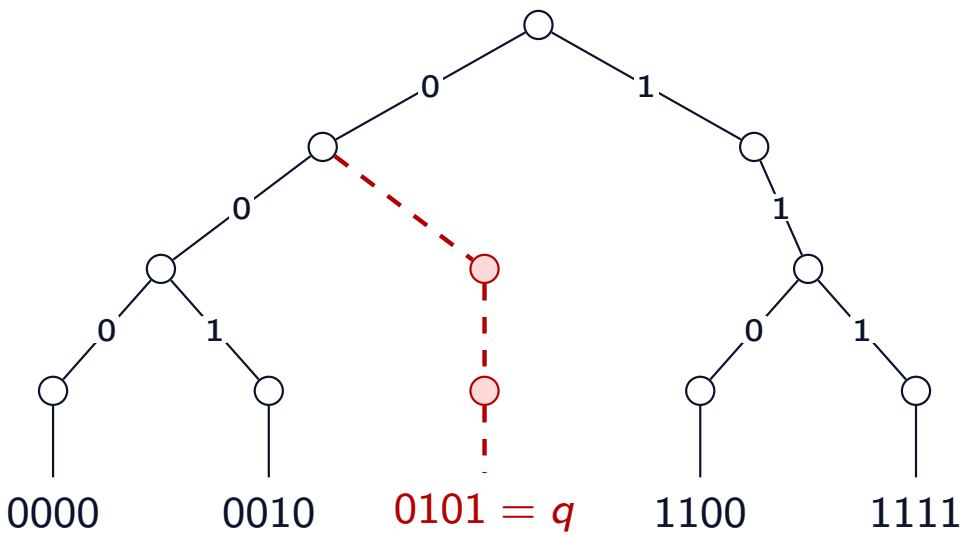
Las llaves 0000, 0010, 1100, 1111 tienen sketches 00, 01, 10, 11 (bits 0 y 2 desde la izquierda). Buscamos a $q = 0101$.



El problema

q puede no ser una llave

Las llaves 0000, 0010, 1100, 1111 tienen sketches 00, 01, 10, 11 (bits 0 y 2 desde la izquierda). Buscamos a $q = 0101$.



$\mathrm{sk}(q) = 00$ lo ubica junto a 0000. ¿Es 0000 el predecesor de 0101?

El sketch de q puede mentir

No: el predecesor es 0010 y el sucesor 1100

q se separa del trie en un nodo que **no** es de ramificación (el prefijo 01 no existe), así que sus bits en las posiciones c_j no dicen en qué lugar de la ordenación cae. El sketch es correcto para las llaves, pero no para consultas arbitrarias.

El sketch de q puede mentir

No: el predecesor es 0010 y el sucesor 1100

q se separa del trie en un nodo que **no** es de ramificación (el prefijo 01 no existe), así que sus bits en las posiciones c_j no dicen en qué lugar de la ordenación cae. El sketch es correcto para las llaves, pero no para consultas arbitrarias.

La corrección

Hay que averiguar **dónde** sale q del trie y buscar con otra palabra, e, cuya posición sí se puede decidir con sketches.

El prefijo común más largo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Sea p el prefijo más largo de q que lo es de alguna llave

Todas las llaves con prefijo p forman un bloque B. Ninguna llave tiene prefijo $p\beta$, con β el bit siguiente de q. Entonces todo B comparte el prefijo $p\bar{\beta}$.

El prefijo común más largo

Sea p el prefijo más largo de q que lo es de alguna llave

Todas las llaves con prefijo p forman un bloque B. Ninguna llave tiene prefijo $p\beta$, con β el bit siguiente de q. Entonces todo B comparte el prefijo $p\bar{\beta}$.

Si $\beta = 1$, todo B es menor que q; si $\beta = 0$, todo B es mayor

- $\beta = 1$: $\mathrm{pred}(q) = \max B$. Sea $e = p01\cdots1$.
- $\beta = 0$: $\mathrm{suc}(q) = \min B$. Sea $e = p10\cdots0$.

e es la palabra con prefijo $p\bar{\beta}$ más cercana a q: se decide solo con p y β.

¿Por qué e sí se puede buscar con sketches?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Dónde queda $\mathrm{sk}(e)$ respecto de los sketches de las llaves?

¿Por qué e sí se puede buscar con sketches?

¿Dónde queda $\mathrm{sk}(e)$ respecto de los sketches de las llaves?

Dentro de B es el máximo (o el mínimo) y fuera de B, queda en su sitio

- Una llave $z \in B$ coincide con e hasta $p\bar{\beta}$; después e tiene solo unos (o ceros), así que $\mathrm{sk}(z) \leq \mathrm{sk}(e)$ (o $\geq$).
- Una llave fuera de B difiere de p en una posición *dentro* de p: allí ramifica el trie, así que el sketch ya la separa y e coincide con p en ese bit.

Por tanto, con $\beta = 1$: $\mathrm{pred}(q)$ es la mayor llave con $\mathrm{sk} \leq \mathrm{sk}(e)$. Con $\beta = 0$: $\mathrm{suc}(q)$ es la menor con $\mathrm{sk} \geq \mathrm{sk}(e)$.

Un ejemplo: $q = 0101$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$p = 0$ y $\beta = 1$

El prefijo común más largo de q con las llaves es 0 (con 0000 y 0010). El bit siguiente de q es 1: $e = p\,0\,1\,1 = 0011$.

Un ejemplo: $q = 0101$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$p = 0$ y $\beta = 1$

El prefijo común más largo de q con las llaves es 0 (con 0000 y 0010). El bit siguiente de q es 1: $e = p\,0\,1\,1 = 0011$.

- $\mathrm{sk}(e) = 01$ (bits 0 y 2 desde la izquierda: 0 y 1).

Un ejemplo: $q = 0101$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$p = 0$ y $\beta = 1$

El prefijo común más largo de q con las llaves es 0 (con 0000 y 0010). El bit siguiente de q es 1: $e = p\,0\,1\,1 = 0011$.

- $\mathrm{sk}(e) = 01$ (bits 0 y 2 desde la izquierda: 0 y 1).
- Llaves con $\mathrm{sk} \leq 01$: 0000 y 0010. La mayor es $0010 = \mathrm{pred}(q)$, y el sucesor es la siguiente llave: 1100.

¿Cómo hallar p sin mirar todas las llaves?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Cuáles llaves pueden tener el prefijo común más largo con q?

¿Cómo hallar p sin mirar todas las llaves?

¿Cuáles llaves pueden tener el prefijo común más largo con q?

Una de las dos *vecinas por sketch* de q

Sean $\mathrm{sk}(x_i) \leq \mathrm{sk}(q) < \mathrm{sk}(x_{i+1})$. El sketch de q coincide con el de B en los bits de ramificación que están dentro de p, y B ocupa un tramo contiguo en la ordenación: $\mathrm{sk}(q)$ cae dentro del tramo de B o justo a su lado. Entonces x_i o x_{i+1} está en B.

La que comparte más prefijo con q

Para $y \in B$, q y y difieren por primera vez en el bit siguiente a p. Para las llaves de fuera, antes. Gana la de menor $q \oplus y$; su bit más significativo da $|p|$.

La búsqueda en un nodo

Algoritmo 1: PREDECESOR-NODO(q) — índice de la mayor llave $\leq q$

$i \leftarrow$ (cantidad de llaves con $\mathrm{sk} \leq \mathrm{sk}(q)) - 1$ // rango por sketch
$y \leftarrow$ la de x_i, x_{i+1} que minimiza $q \oplus y$
si $y = q$ **entonces devolver** el índice de y
$j \leftarrow \mathrm{msb}(q \oplus y)$, $\beta \leftarrow$ el bit j de q // aquí se separan
si $\beta = 1$ **entonces**
  $e \leftarrow (q \gg (j+1)) \ll (j+1) \mid (2^j - 1)$ // $e = p\,0\,1\cdots1$
  devolver (cantidad de llaves con $\mathrm{sk} \leq \mathrm{sk}(e)) - 1$
en otro caso
  $e \leftarrow (q \gg (j+1)) \ll (j+1) \mid 2^j$ // $e = p\,1\,0\cdots0$
  devolver (cantidad de llaves con $\mathrm{sk} < \mathrm{sk}(e)) - 1$

La búsqueda en un nodo

Algoritmo 2: PREDECESOR-NODO(q) — índice de la mayor llave $\leq q$

$i \leftarrow$ (cantidad de llaves con $\mathrm{sk} \leq \mathrm{sk}(q)) - 1$ `// rango por sketch`
$y \leftarrow$ la de x_i, x_{i+1} que minimiza $q \oplus y$
si $y = q$ **entonces devolver** el índice de y
$j \leftarrow \mathrm{msb}(q \oplus y)$, $\beta \leftarrow$ el bit j de q `// aquí se separan`
si $\beta = 1$ **entonces**
  $e \leftarrow (q \gg (j+1)) \ll (j+1) \mid (2^j - 1)$ `//` $e = p\,0\,1\cdots1$
  devolver (cantidad de llaves con $\mathrm{sk} \leq \mathrm{sk}(e)) - 1$
en otro caso
  $e \leftarrow (q \gg (j+1)) \ll (j+1) \mid 2^j$ `//` $e = p\,1\,0\cdots0$
  devolver (cantidad de llaves con $\mathrm{sk} < \mathrm{sk}(e)) - 1$

Faltan tres ingredientes en $O(1)$: calcular $\mathrm{sk}(q)$, contar llaves con sketch menor o igual, y msb.

Sketch con una multiplicación

El sketch perfecto es caro; el aproximado, no

UTEC
UNIVERSIDAD DE INGENIERÍA

Idea

Con una multiplicación se copia la palabra desplazada varias veces, de modo que cada bit importante caiga en un lugar conocido:

$$(x \;\&\; M) \cdot \sum_j 2^{m_j} = \sum_i \sum_j x_{c_i} \, 2^{c_i + m_j}, \qquad M = \sum_i 2^{c_i}.$$

¿Qué hay que exigirle a los m_j?

El sketch perfecto es caro; el aproximado, no

UTEC
UNIVERSIDAD DE INGENIERÍA

Idea

Con una multiplicación se copia la palabra desplazada varias veces, de modo que cada bit importante caiga en un lugar conocido:

$$(x \,\&\, M) \cdot \sum_j 2^{m_j} = \sum_i \sum_j x_{c_i} \, 2^{c_i + m_j}, \qquad M = \sum_i 2^{c_i}.$$

¿Qué hay que exigirle a los m_j?

Tres propiedades

1. Los r^2 exponentes $c_i + m_j$ son distintos: sin acarreos.
2. Los $P_j = c_j + m_j$ crecen con j: se conserva el orden.
3. $P_{r-1} - P_0 = O(r^4) = O(w^{4/5})$: queda compacto.

Elegir los m_j

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Paso 1: sin colisiones

Elegir $t_0, t_1, \ldots < r^3$ tales que $t_s \not\equiv t_i + c_a - c_b \pmod{r^3}$ para $i < s$ y todo a, b. Cada t_s debe evitar menos de $s\,r^2 < r^3$ residuos, así que siempre existe.

Paso 2: en ventanas crecientes

Sea m_j el valor congruente con t_j módulo r^3 tal que

$$P_j = c_j + m_j \in [\,w + j\,r^3,\ w + (j+1)\,r^3\,).$$

Elegir los m_j

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Paso 1: sin colisiones

Elegir $t_0, t_1, \ldots < r^3$ tales que $t_s \not\equiv t_i + c_a - c_b \pmod{r^3}$ para $i < s$ y todo a, b. Cada t_s debe evitar menos de $s\,r^2 < r^3$ residuos, así que siempre existe.

Paso 2: en ventanas crecientes

Sea m_j el valor congruente con t_j módulo r^3 tal que

$$P_j = c_j + m_j \in [\,w + j\,r^3,\ w + (j+1)\,r^3\,).$$

Las tres propiedades se cumplen

Distintos: si $j \neq s$ coincidieran, $t_s \equiv t_j + c_i - c_a$; si $j = s$, $c_i \neq c_a$.
Crecientes: ventanas disjuntas. Compacto: $P_{r-1} - P_0 < r \cdot r^3$.

Un ejemplo con $r = 3$

$w = 6$, bits importantes $c = (0, 1, 3)$, $r^3 = 27$

$t = (0, 4, 8)$ da $m = (27, 58, 62)$. Los 9 exponentes $c_i + m_j$:

	$m_0 = 27$	$m_1 = 58$	$m_2 = 62$
$c_0 = 0$	27	58	62
$c_1 = 1$	28	59	63
$c_2 = 3$	30	61	65

Un ejemplo con $r = 3$

$w = 6$, bits importantes $c = (0, 1, 3)$, $r^3 = 27$

$t = (0, 4, 8)$ da $m = (27, 58, 62)$. Los 9 exponentes $c_i + m_j$:

	$m_0 = 27$	$m_1 = 58$	$m_2 = 62$
$c_0 = 0$	27	58	62
$c_1 = 1$	28	59	63
$c_2 = 3$	30	61	65

- Distintos y las diagonales $P = (27, 59, 65)$ caen en las ventanas $[6, 33), [33, 60), [60, 87)$.
- Para $x = 010011$: $x \,\&\, M$ tiene los bits c_0, c_1; tras multiplicar y enmascarar con $D = 2^{27} + 2^{59} + 2^{65}$ quedan los bits 27 y 59: el sketch **aproximado** es $1 + 2^{32}$.

El sketch aproximado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$\widetilde{\mathrm{sk}}(x) = (((x \,\&\, M) \cdot m) \,\&\, D) \gg P_0$

con $m = \sum_j 2^{m_j}$. Son una conjunción, una multiplicación, otra conjunción y un desplazamiento: $O(1)$.

El sketch aproximado

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$$\widetilde{\mathrm{sk}}(x) = (((x \,\&\, M) \cdot m) \,\&\, D) \gg P_0$$

con $m = \sum_j 2^{m_j}$. Son una conjunción, una multiplicación, otra conjunción y un desplazamiento: $O(1)$.

Conserva el orden de las llaves (y de cualquier q)

Es el mismo sketch con los bits separados en posiciones crecientes: comparar dos aproximados equivale a comparar los perfectos. Mide a lo más r^4 bits.

Suponemos que el producto de dos palabras se obtiene completo ($2w$ bits), como en la instrucción de multiplicación de un procesador.

Comparación en paralelo

La palabra del nodo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una palabra con todos los sketches

Cada llave ocupa un bloque de $r^4 + 1$ bits: un 1 delante (el *centinela*) y su sketch aproximado. Con k llaves, el nodo ocupa $k(r^4 + 1) = O(w)$ bits.

La palabra del nodo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Una palabra con todos los sketches

Cada llave ocupa un bloque de $r^4 + 1$ bits: un 1 delante (el *centinela*) y su sketch aproximado. Con k llaves, el nodo ocupa $k(r^4 + 1) = O(w)$ bits.

Para comparar, una sola resta

Se replica $\widetilde{\mathrm{sk}}(q)$ en k bloques (multiplicando por $0\cdots01\,0\cdots01\cdots$) y se resta de la palabra del nodo. En cada bloque, $1\mathrm{sk}(x_i) - 0\mathrm{sk}(q) > 0$: no hay préstamos entre bloques, y el centinela sobrevive si y solo si $\mathrm{sk}(x_i) \geq \mathrm{sk}(q)$.

Un ejemplo con sketches de 3 bits

	x_0	x_1	x_2	x_3
Sketches	000	001	011	111
Nodo	1000	1001	1011	1111
$\mathrm{sk}(q) = 010$ replicado	0010	0010	0010	0010

Un ejemplo con sketches de 3 bits

	x_0	x_1	x_2	x_3
Sketches	000	001	011	111
Nodo	1000	1001	1011	1111
$\mathrm{sk}(q) = 010$ replicado	0010	0010	0010	0010
Resta bloque a bloque	0110	0111	1001	1101

Los centinelas $0, 0, 1, 1$: dos llaves tienen sketch menor que $\mathrm{sk}(q)$

q cae entre x_1 y x_2: $001 < 010 < 011$.

Contar los unos

Los centinelas quedaron separados cada $r^4 + 1$ bits. ¿Cómo sumarlos con una multiplicación?

Contar los unos

Los centinelas quedaron separados cada $r^4 + 1$ bits. ¿Cómo sumarlos con una multiplicación?

Multiplicando por $0 \cdots 01\, 0 \cdots 01 \cdots 0 \cdots 01$

Cada centinela se copia en todos los bloques; en el bloque de más a la izquierda llegan los k centinelas, uno por copia, y ahí se lee la suma (cabe en un bloque porque $k < 2^{r^4}$). Se enmascara y se desplaza: $O(1)$.

Resultado

En $O(1)$: **la cantidad de llaves con** sk **mayor o igual que el de** q, es decir el rango de q entre los sketches. Con $\mathrm{sk}(q) + 1$ se obtiene el $\leq$.

El bit más significativo

Falta msb: el primer bit donde difieren q e y

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$\mathrm{msb}(q \oplus y)$ en $O(1)$

Parte la palabra en $\sqrt{w}$ clusters de $\sqrt{w}$ bits.

1. Obtener, con unas pocas operaciones de palabra, un bit por cada cluster no vacío.
2. Comprimir esos $\sqrt{w}$ bits con *otro* sketch (una multiplicación).
3. Comparar en paralelo contra las potencias de 2: eso da el primer cluster no vacío.
4. Repetir lo mismo dentro de ese cluster.

El paso 1, con $w = 16$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

x	0101	0000	1000	1101
F (primer bit de cada cluster)	1000	1000	1000	1000
$t_1 = x \,\&\, F$	0000	0000	1000	1000

El paso 1, con $w = 16$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

x	0101	0000	1000	1101
F (primer bit de cada cluster)	1000	1000	1000	1000
$t_1 = x \,\&\, F$	0000	0000	1000	1000
$t_2 = x \oplus t_1$ (sin los primeros bits)	0101	0000	0000	0101
$t_3 = F - t_2$	0011	1000	1000	0011

El paso 1, con $w = 16$

x	0101	0000	1000	1101
F (primer bit de cada cluster)	1000	1000	1000	1000
$t_1 = x \,\&\, F$	0000	0000	1000	1000
$t_2 = x \oplus t_1$ (sin los primeros bits)	0101	0000	0000	0101
$t_3 = F - t_2$	0011	1000	1000	0011
$t_4 = (t_3 \oplus F) \,\&\, F$	1000	0000	0000	1000
$y = t_1 \mid t_4$	1000	0000	1000	1000

y tiene un 1 en la cabeza de cada cluster *no vacío*. En la resta, el primer bit de F se pierde en un cluster si y solo si quedaba algo más en él.

El resto de msb

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Compactar y comparar

Los bits de y están en las posiciones $\sqrt{w} - 1 + i\sqrt{w}$. Multiplicando por $\sum_j 2^{w-(\sqrt{w}-1)-j\sqrt{w}+j}$ los $\sqrt{w}$ bits caen consecutivos desde la posición w: un sketch perfecto de $\sqrt{w}$ bits.

El resto de msb

Compactar y comparar

Los bits de y están en las posiciones $\sqrt{w} - 1 + i\sqrt{w}$. Multiplicando por $\sum_j 2^{w-(\sqrt{w}-1)-j\sqrt{w}+j}$ los $\sqrt{w}$ bits caen consecutivos desde la posición w: un sketch perfecto de $\sqrt{w}$ bits.

Después, comparación en paralelo con $1, 2, 4, \ldots, 2^{\sqrt{w}-1}$

Las $\sqrt{w}$ potencias, con un bloque de $\sqrt{w}+1$ bits cada una, ocupan menos de $2w$ bits: una restas y un conteo dan el cluster más significativo, y repetir dentro de él da el bit. Todo es $O(1)$.

El fusion tree

Todo junto

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema (Fredman y Willard)

Un conjunto estático de n palabras de w bits se guarda en un B-tree de ramificación $w^{1/5}$ con $O(n)$ de espacio, y PREDECESOR y SUCESOR cuestan $O(\log_w n)$ en la word RAM.

Todo junto

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Teorema (Fredman y Willard)

Un conjunto estático de n palabras de w bits se guarda en un B-tree de ramificación $w^{1/5}$ con $O(n)$ de espacio, y PREDECESOR y SUCESOR cuestan $O(\log_w n)$ en la word RAM.

Por qué

La altura es $O(\log_w n)$ y en cada nodo la búsqueda es $O(1)$: sketch aproximado (una multiplicación), rango por comparación en paralelo (una resta y una multiplicación), msb, y otra pasada con e.

Versiones dinámicas: árboles exponenciales ($O(\log_w n + \lg\lg n)$ por actualización, determinista) y con hashing ($O(\log_w n)$ esperado). Sin multiplicación (modelo AC^0) se puede calcular el sketch perfecto con otra técnica.

El panorama

Lo mejor de dos mundos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

vEB y Y-fast: $O(\lg w)$. Fusion tree: $O(\log_w n)$.

El predecesor cuesta $O(\text{mín}\{\lg w,\ \log_w n\})$.

Lo mejor de dos mundos

vEB y Y-fast: $O(\lg w)$. Fusion tree: $O(\log_w n)$.

El predecesor cuesta $O(\text{mín}\{\lg w,\ \log_w n\})$.

$\text{mín}\{\lg w, \log_w n\} = O(\sqrt{\lg n})$

Si $\lg w \leq \sqrt{\lg n}$, ya está. Si no, $\lg w > \sqrt{\lg n}$ y

$$\log_w n = \frac{\lg n}{\lg w} < \frac{\lg n}{\sqrt{\lg n}} = \sqrt{\lg n}.$$

¿Es el mejor tiempo posible?

Cotas inferiores

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

En cell probe, para espacio polinomial

El predecesor estático necesita

$$\Omega\left(\text{mín} \left\{ \log_w n,\ \frac{\lg w}{\lg \lg n} \right\} \right)$$

accesos a memoria (técnica de *round elimination*: Miltersen, Nisan, Safra, Wigderson; Beame y Fich).

Cotas inferiores

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

En cell probe, para espacio polinomial

El predecesor estático necesita

$$\Omega\left(\min\left\{\log_w n,\ \frac{\lg w}{\lg\lg n}\right\}\right)$$

accesos a memoria (técnica de *round elimination*: Miltersen, Nisan, Safra, Wigderson; Beame y Fich).

La mejor estructura de arriba es óptima, salvo un factor $\lg\lg$

vEB / Y-fast para w polilogarítmico en n y fusion trees para w grande, justo en el rango donde cada una gana.

Ordenar enteros

Método	Tiempo	Condición
Comparaciones	$O(n \lg n)$	ninguna
Counting sort	$O(n + 2^w)$	$w = O(\lg n)$: $O(n)$
Radix sort	$O(n\, w / \lg n)$	$w = O(\lg n)$: $O(n)$
Ordenar con vEB / Y-fast	$O(n \lg w)$	$O(n \lg \lg n)$ si $w = \lg^{O(1)} n$
Signature sort	$O(n)$	$w = \Omega(\lg^{2+\epsilon} n)$
Han	$O(n \lg \lg n)$	determinista
Han y Thorup	$O(n\sqrt{\lg \lg n})$	aleatorizado

¿Se puede ordenar n enteros en $O(n)$ para todo w?

Ordenar enteros

Método	Tiempo	Condición
Comparaciones	$O(n \lg n)$	ninguna
Counting sort	$O(n + 2^w)$	$w = O(\lg n)$: $O(n)$
Radix sort	$O(n\, w / \lg n)$	$w = O(\lg n)$: $O(n)$
Ordenar con vEB / Y-fast	$O(n \lg w)$	$O(n \lg \lg n)$ si $w = \lg^{O(1)} n$
Signature sort	$O(n)$	$w = \Omega(\lg^{2+\epsilon} n)$
Han	$O(n \lg \lg n)$	determinista
Han y Thorup	$O(n\sqrt{\lg \lg n})$	aleatorizado

¿Se puede ordenar n enteros en $O(n)$ para todo w?

Es un problema abierto (y, por la equivalencia, también una cola de prioridad con operaciones $O(1)$)

Signature sort, en una frase por paso

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Para $w = \Omega(\lg^{2+\epsilon} n)$

1. Partir cada entero en $\lg^{\epsilon} n$ bloques (dígitos).
2. Reemplazar cada bloque por un hash perfecto de $O(\lg n)$ bits: hay pocos bloques distintos.
3. Con palabras tan cortas, el *packed sort* (mergesort con redes de ordenamiento bitónicas sobre palabras) ordena en $O(n)$.
4. Un trie comprimido sobre los hashes; ordenar sus aristas recursivamente da el orden real; el recorrido en inorden entrega los enteros ordenados.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Fusion tree: B-tree de ramificación $w^{1/5}$; si cada nodo se busca en $O(1)$, el predecesor cuesta $O(\log_w n)$ con $O(n)$ de espacio.
- Sketch: los $k-1$ bits donde ramifica el trie bastan para ordenar las llaves, y los sketches de un nodo caben en una palabra. Con una q cualquiera se corrige con e y el prefijo común más largo.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Fusion tree: B-tree de ramificación $w^{1/5}$; si cada nodo se busca en $O(1)$, el predecesor cuesta $O(\log_w n)$ con $O(n)$ de espacio.
- Sketch: los $k-1$ bits donde ramifica el trie bastan para ordenar las llaves, y los sketches de un nodo caben en una palabra. Con una q cualquiera se corrige con e y el prefijo común más largo.
- Con una multiplicación se calcula el sketch; con una resta se compara con todas las llaves a la vez; msb también es $O(1)$.
- $\min\{\lg w, \log_w n\} = O(\sqrt{\lg n})$, óptimo salvo $\lg\lg$ en cell probe. Ordenar enteros en tiempo lineal para todo w sigue abierto.

Logros de la sesión



Logros de la sesión

- Sabemos por qué un B-tree de ramificación $w^{1/5}$ da $O(\log_w n)$ si el nodo se resuelve en $O(1)$
- Sabemos definir el sketch perfecto, demostrar que conserva el orden y corregir la búsqueda de una q cualquiera

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos por qué un B-tree de ramificación $w^{1/5}$ da $O(\log_w n)$ si el nodo se resuelve en $O(1)$
- Sabemos definir el sketch perfecto, demostrar que conserva el orden y corregir la búsqueda de una q cualquiera
- Sabemos calcular el sketch con una multiplicación y comparar en paralelo con una resta
- Sabemos combinar vEB y fusion trees en $O(\sqrt{\lg n})$ y ubicar el predecesor y el ordenamiento de enteros en el panorama

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias

Referencias

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Bibliografía citada

- Fredman, M. L., & Willard, D. E. (1993). Surpassing the information theoretic barrier with fusion trees. *Journal of Computer and System Sciences*, 47, 424–436.
- Andersson, A., Miltersen, P. B., & Thorup, M. (1999). Fusion trees can be implemented with AC^0 instructions only. *Theoretical Computer Science*, 215, 337–344.
- Andersson, A., Hagerup, T., Nilsson, S., & Raman, R. (1998). Sorting in linear time? *Journal of Computer and System Sciences*, 57, 74–93.
- Demaine, Erik (2012). *6.851: Advanced Data Structures (Spring'12)*, Lectures 12, 13 y 14. `https://courses.csail.mit.edu/6.851/spring12/`.

Agradecimientos

Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.